

Assignment 1 evidence

Yihao Yin (yihaoyin)

generated 2026-08-27 22:14 ????? on Windows-11-10.0.26200-SP0

project sensorlab, uv.lock sha256 c082196efbe47208

evidence script sha256 7ab84f0c7ac1349c2c02e239f004ac9e34e1d39e7892bdf0e3c087ca84b3736d

Score **1.00** (6 of 6 checks passed)

What this script found

```
entry point    sensorlab.train
package        src\sensorlab
notebook       notebooks\explore.ipynb
README         README.md
MLflow store   sqlite:///mlflow.db
```

Definition of done, 6 of 6

- [PASS] The environment rebuilds from the lockfile
- [PASS] The same seed prints the same number twice
- [PASS] A different seed prints a different number
- [PASS] The raw data sits under data/ and nowhere else in the project
- [PASS] The notebook's cells ran in order from a fresh kernel
- [PASS] The metric printed by the seed 0 run also appears in the MLflow table

Each line is decided from the output below, not asserted.

Transcript

tool versions

```
$ uv --version [exit 0, 0.0s]
```

```
uv 0.12.6 (7938ca5d5 2026-08-25 x86_64-pc-windows-msvc)
```

rebuild the environment from the lockfile

```
$ uv sync --locked [exit 0, 5.9s]
```

```
Using CPython 3.13.15
Creating virtual environment at: .venv
Resolved 176 packages in 1ms
Installed 169 packages in 5.87s
+ aiohappyeyeballs==2.7.1
+ aiohttp==3.14.3
+ aiosignal==1.4.0
+ alembic==1.19.1
+ annotated-doc==0.0.5
+ annotated-types==0.8.0
+ anyio==4.14.2
+ argon2-cffi==25.1.0
+ argon2-cffi-bindings==26.1.0
+ arrow==1.4.0
+ [...] 151 lines elided [...]
+ wcwidth==0.8.2
+ webcolors==25.10.0
+ webencodings==0.6.1
+ websocket-client==1.9.0
+ werkzeug==3.1.8
+ widgetsnbextension==4.0.16
+ yarl==1.24.5
+ zipp==4.1.0
```

resolved versions

```
$ uv run python -c "..." [exit 0, 6.4s]
```

```
python 3.13.15
numpy 2.5.2
```

```
pandas 2.3.3
sklearn 1.9.0
mlflow 3.15.2
```

run, seed 0

```
$ uv run python -m sensorlab.train --seed 0 [exit 0, 6.6s]
```

```
r2=0.7755448891992244
slope 0.00576, intercept -4.27356
```

run, seed 0 again

```
$ uv run python -m sensorlab.train --seed 0 [exit 0, 3.9s]
```

```
r2=0.7755448891992244
slope 0.00576, intercept -4.27356
```

run, seed 1

```
$ uv run python -m sensorlab.train --seed 1 [exit 0, 3.9s]
```

```
r2=0.7576898018166922
slope 0.00575, intercept -4.26084
```

MLflow runs

```
$ uv run python -c "... [exit 0, 2.2s]
```

run_id	params.seed	metrics.r2		start_time
50c98c1864984b73b33f654ff8d80dfb			1	0.757690 2026-08-28 02:14:41.376000+00:00
c2b0b8ee2f05496fb999698d28acf8			0	0.775545 2026-08-28 02:14:37.481000+00:00
b1821b9ad1714ba086476adafc635b8a			0	0.775545 2026-08-28 02:14:33.510000+00:00
940f33c7ae1e4fd58e2cd15b3f15a68e			1	0.757690 2026-08-28 02:09:43.811000+00:00
6ad7fa964b6d47f08dae20f21885bf9d			0	0.775545 2026-08-28 02:09:40.171000+00:00
c09b7b88c7f7492da94f8c1b6f68c660			0	0.775545 2026-08-28 02:09:36.553000+00:00
1e9a8e3891bc442a863d977da1101824			0	0.775545 2026-08-28 01:39:49.237000+00:00
eb28d237cbae484f9945a6af058d4043			2	0.775716 2026-08-28 01:21:13.684000+00:00
97e98c2dee5942d6a1c20d84af3ac681			1	0.757690 2026-08-28 01:20:27.970000+00:00
5f4e0ed28749469e8b22897f9993b381			0	0.775545 2026-08-28 01:16:09.409000+00:00

.gitignore

```
.venv/
data/
mlruns/
mlartifacts/
mlflow.db

__pycache__/
*.py[cod]
.ipynb_checkpoints/
```

pyproject.toml

```
[project]
name = "sensorlab"
version = "0.1.0"
description = "Add your description here"
readme = "README.md"
requires-python = ">=3.11"
dependencies = [
    "matplotlib>=3.11.1",
    "mlflow>=3.15.2",
    "pandas>=2.3.3",
    "scikit-learn>=1.9.0",
]

[project.scripts]
sensorlab = "sensorlab:main"
```

```
[build-system]
requires = ["uv_build>=0.12.6,<0.13.0"]
build-backend = "uv_build"

[dependency-groups]
dev = [
    "jupyter>=1.1.1",
]
```

The package

```
--- src\sensorlab\__init__.py
def main() -> None:
    print("Hello from sensorlab!")

--- src\sensorlab\dataprocess.py
import numpy as np
import pandas as pd
import io
import urllib.request
import zipfile
from pathlib import Path

URL = 'https://archive.ics.uci.edu/static/public/360/air+quality.zip'

def load(path):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    if not path.exists():
        print(f'fetching {URL}')
        with urllib.request.urlopen(URL) as response:
            payload = response.read()
        with zipfile.ZipFile(io.BytesIO(payload)) as archive:
            path.write_bytes(archive.read('AirQualityUCI.csv'))

    return pd.read_csv(path, sep=';', decimal=',')

def clean(df):
    df = (df.dropna(axis=1, how='all').dropna(how='all'))

    df['ts'] = pd.to_datetime(
        df['Date'] + ' ' + df['Time'].str.replace('.', ':', regex=False),
        format='%d/%m/%Y %H:%M:%S',
    )
    df = df.replace(-200, np.nan)

    return df

--- src\sensorlab\train.py
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score
from .dataprocess import clean, load
from pathlib import Path
import mlflow
import argparse

def train(data_path, seed):
    df = load(data_path)
    df = clean(df)
```

```

d = df.dropna(subset=['CO(GT)', 'PT08.S1(CO)'])

X = d[['PT08.S1(CO)']]
y = d['CO(GT)']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=seed)

model = LinearRegression().fit(X_train, y_train)
score = r2_score(y_test, model.predict(X_test))

return model, score

# Argument Part is generated by ChatGPT
def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--seed", type=int, default=0)
    parser.add_argument(
        "--data-path",
        default="data/AirQualityUCI.csv",
    )
    args = parser.parse_args()

    model, score = train(args.data_path, args.seed)

    mlflow.set_tracking_uri("sqlite:///mlflow.db")
    mlflow.set_experiment("sensorlab")

    with mlflow.start_run(run_name=f"seed-{args.seed}"):
        mlflow.log_params({"seed": args.seed, "data": Path(args.data_path).name})
        mlflow.log_metric("r2", score)

    print(f"r2={score}")
    print(f"slope {model.coef_[0]:.5f}, "f"intercept {model.intercept_: .5f}")

if __name__ == '__main__':
    main()

```

Notebook code cells: notebooks\explore.ipynb

```

In [1]:
# Run this first on Colab. Anywhere else this cell does nothing.
#
# Only genuinely missing packages are installed, so Colab's own versions of
# everything it already ships are left alone.
#
# Generated by tools/colab_setup.py from this notebook's imports. Edit that.
import importlib.util
import subprocess
import sys

REQUIREMENTS = {
    "matplotlib": "matplotlib",
    "numpy": "numpy",
    "pandas": "pandas",
    "sklearn": "scikit-learn",
}

def _missing(module):
    try:
        return importlib.util.find_spec(module) is None
    except ModuleNotFoundError: # the parent package is absent
        return True

if "google.colab" in sys.modules:

```

```

need = sorted({pip for mod, pip in REQUIREMENTS.items() if _missing(mod)})
if need:
    print("installing:", " ".join(need))
    subprocess.run([sys.executable, "-m", "pip", "install", "-q", *need], check=True)
print("Colab setup done." if need else "Colab: nothing to install.")

```

```

In [2]:
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sensorlab.dataprocess import clean, load
from sensorlab.train import train

print('numpy ', np.__version__)
print('pandas', pd.__version__)

```

```

In [3]:
from pathlib import Path

DATA_PATH = Path("../data/AirQualityUCI.csv")

print(f'using {DATA_PATH}')

```

```

In [4]:
df = clean(load(DATA_PATH))

print(f'{len(df)} rows, {df.ts.min().date()} to {df.ts.max().date()}')
df.head(3)

```

```

In [5]:
d = df.dropna(subset=['CO(GT)', 'PT08.S1(CO)'])

fig, ax = plt.subplots(figsize=(6, 4.2))
ax.scatter(d['PT08.S1(CO)'], d['CO(GT)'], s=4, alpha=0.25)
ax.set_xlabel('PT08.S1(CO) sensor response')
ax.set_ylabel('CO(GT) reference, mg/m$^3$')
ax.set_title('Sensor response vs reference measurement')
plt.show()

```

```

In [6]:
seed = 0
model, score = train(DATA_PATH, seed)

print(f'R2 on held-out data: {score:.4f}')
print(f'slope {model.coef_[0]:.5f}, intercept {model.intercept_:.5f}')

```

```

In [7]:
hourly_co = d.set_index('ts')['CO(GT)'].resample('h').mean().dropna()
hours_co = (hourly_co.index - hourly_co.index[0]).total_seconds() / 3600.0

exposure_co = np.trapezoid(hourly_co.values, hours_co)
print(f'Integrated CO exposure: {exposure_co:,.0f} mg-h/m3')

```

```

In [8]:
nox = df.dropna(subset=['NOx(GT)'])
hourly_nox = nox.set_index('ts')['NOx(GT)'].resample('h').mean().dropna()
hours_nox = (hourly_nox.index - hourly_nox.index[0]).total_seconds() / 3600.0

exposure_nox = np.trapezoid(hourly_nox.values, hours_nox)
print(f'Integrated NOx exposure: {exposure_nox:,.0f} ppb-h')

```

README: README.md

SensorLab: reproducible air quality calibration

1. Project purpose

A sensor analysis can produce a reasonable result and still be difficult to reproduce. The result may change when the package versions, data path, or train/test split changes. This project rebuilds a simple air-quality calibration analysis as a Python package and controls those sources of variation. 'uv' records the environment, all paths are relative to the project, the split takes an explicit seed, and MLflow keeps the run history. Consequently, a reported result can be traced to the code, data, and seed that produced it.

The analysis uses the 'PT08.S1(CO)' sensor response to predict the reference carbon-monoxide measurement 'CO(GT)' with a linear regression model. The raw data come from the [UCI Air Quality Data Set](https://archive.ics.uci.edu/dataset/360/air-quality).

2. Rebuild the environment

Install 'uv' and open a terminal in the project root. Then run:

```
'''bash
uv sync
'''
```

The project pins its Python version in '.python-version' and records the exact package resolution in 'uv.lock'. Deleting '.venv' and running 'uv sync' again rebuilds the environment from these files without relying on packages already installed on the machine.

3. Get the data

The project downloads the data when it is needed. If the requested CSV is absent, 'sensorlab.dataprocess.load()' downloads the UCI archive and writes the raw file to:

```
'''text
data/AirQualityUCI.csv
'''
```

'load()' reads the semicolon-separated file and its comma decimal marks. 'clean()' removes empty rows and columns, constructs the timestamp, and replaces the '-200' missing-value sentinel with 'NaN'. The raw data remain under 'data/' and are excluded from Git.

4. Reproduce the result

From the project root, run:

```
'''bash
uv run python -m sensorlab.train --seed 0
'''
```

The seed '0' run should print:

```
'''text
r2=0.7755448891992244
'''
```

Repeating the command with seed '0' produces the same value. Changing the seed changes the train/test split and should produce a different value, for example:

```
'''bash
uv run python -m sensorlab.train --seed 1
'''
```

MLflow writes the seed, data filename, and 'r2' metric for each run to the local 'mlflow.db' store. To inspect the recorded runs, use:

```
'''bash
uv run mlflow ui --backend-store-uri sqlite:///mlflow.db
```

'''

5. Generative-AI use

I used OpenAI Codex during this assignment to diagnose problems in the environment and code. It also identified that an active VS Code Jupyter kernel was blocking the evidence script from deleting and rebuilding '.venv'. I ran the project locally to check the workflow and am responsible for understanding the final code and results.

Summary line

The script hash should match the checksum published beside the script.

```
{"andrew_id": "yihaoyin", "generated": "2026-08-27 22:14 \u00e1\u00e8\u00f\u00ee\u00f6", "score": 1.0,
  "passed": 6, "of": 6, "failed": [], "script_sha256": "7ab84f0c7ac1349c2c02e239f004ac9e34e1d39e7892bdf0e3c087ca84b3736d", "uv_lock_sha256": "c082196efbe47208", "module": "sensorlab.train"}
```